

CacheBlend

Fast Large Language Model Serving for RAG
with Cached Knowledge Fusion

Presented By: Vanshika Chokhani, Aditya



Introduction

RAG input



Full Prefill

Correct — but TTFT scales with input length.

Too slow for long RAG contexts.

Prefix KV Reuse

Fast — but only valid at position 0.

Non-prefix chunks carry stale KV — cross-chunk attention is missing.

CACHEBLEND

Reuse all KV caches

Selective recompute

Pipeline I/O + compute



Background: LLM Inference & KV Cache

① PREFILL

TTFT — user waits here

PREFILL

the

cat

sat

on

mat

↓ Single forward pass — computes K, V at every layer for every token

KV Cache = concatenation of [K,V] tensors · shape: $2 \times N \times L$

Slow — $O(N \cdot L)$ compute · bottleneck for long inputs

② DECODE (autoregressive)

DECODE

The

refund

takes

5–7

days...

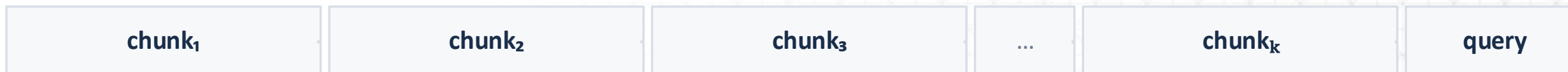
↑ One token per step — attends over cached K, V (no reprocessing)

Reads KV Cache each step

Fast — reuses prefill output, no re-computation

Background: RAG & Precomputed KV Cache

RAG input structure:



Prefilled on every request

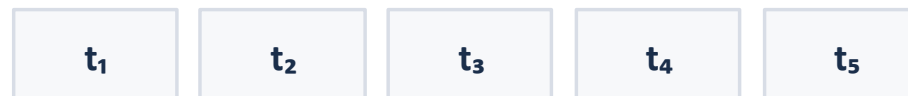
Key Observation

Retrieved chunks are reused across many different requests. The same document appears as a top-K result for hundreds of queries. Its KV cache is recomputed from scratch every single time.

Opportunity: precompute the KV cache for each chunk once and reuse it across all requests that include it.

Why KV reuse only works for the prefix

Each KV tensor encodes attention to ALL preceding tokens:



$\leftarrow t_4$ attends to t_1, t_2, t_3

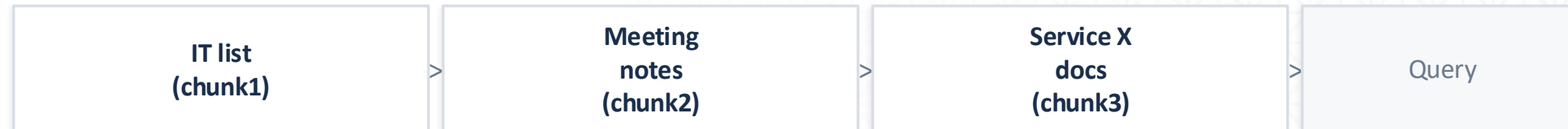


chunk_2 's KV was computed without seeing chunk_1 — cross-chunk attention is missing.

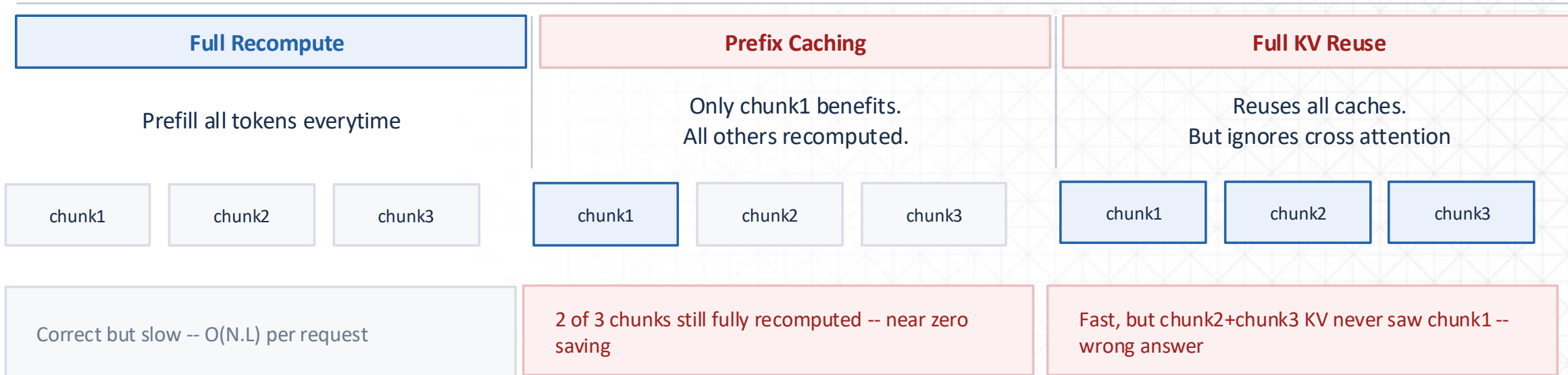
Motivation: Why Existing Approaches Fall Short

Running example — IT department LLM

Scenario: queries like [who proposed using RAG?] and [who graduated from college Y?] both need the same IT employee list.



Same 3 chunks reused across many different queries



Reuse ALL precomputed KV caches AND fix staleness - achieving both speed and quality.

Fast KV Cache Fusing: Selective KV Recompute

The Algorithm

Goal: update $KV_pre \rightarrow KV_new$ to minimise $\delta_{attn}(A_new, A_full)$ on every layer.

Process input layer-by-layer (cannot skip -- hidden states propagate top-down).

At each layer i :

1 Select HKVD tokens

Tokens with highest KV deviation from KV_full - concentrated at chunk boundaries.

2 Recompute K, V for HKVD

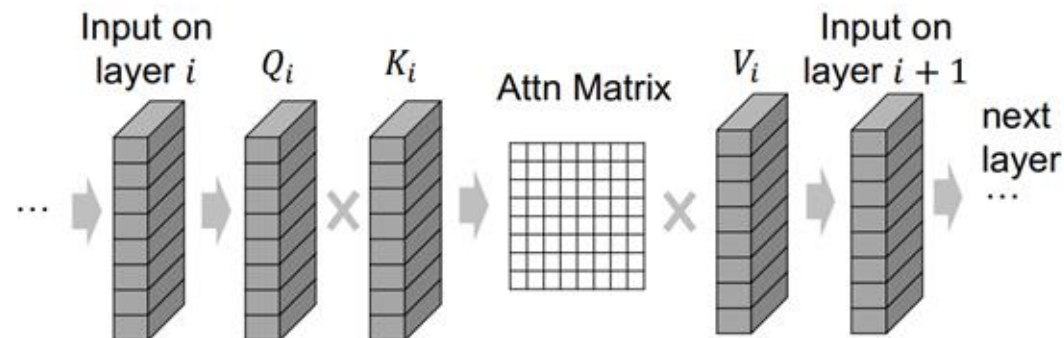
$KV_new_i[j] = KV_full_i[j]$ (fresh compute)

3 Reuse KV_pre for all others

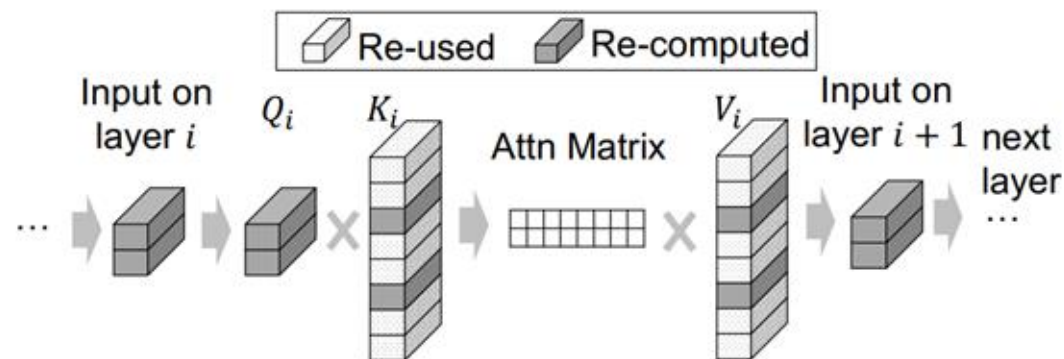
$KV_new_i[j] = KV_pre_i[j]$ (cached, no compute)

Update fraction $|S_I|/N < 15\% \rightarrow$ same quality as full KV recompute (KV_full)

Layer-by-layer execution: HKVD tokens recomputed, rest reuse KV_pre



(a) Full KV recompute for reference

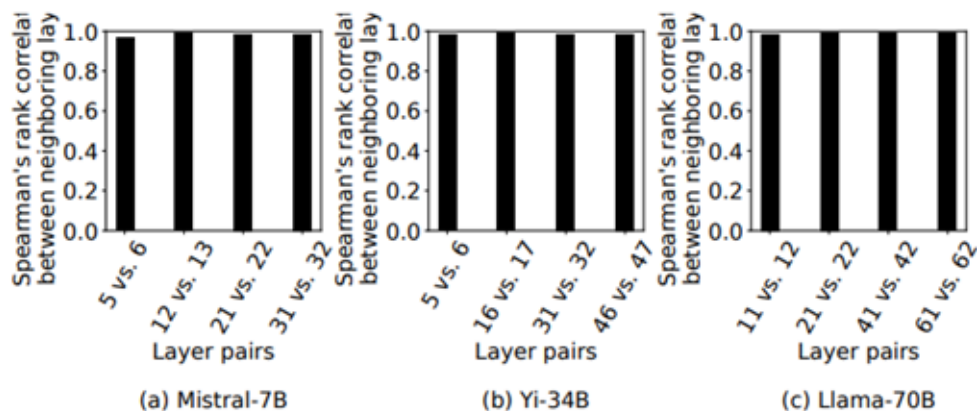


(b) Selective KV recompute on two selected tokens

Fast KV Cache Fusing: Selecting HKVD Tokens

Recomputing higher-deviation tokens reduces Δ_{attn}

Pick the top 10-15% tokens



HKVD tokens on one layer are likely to be HKVD on the next.

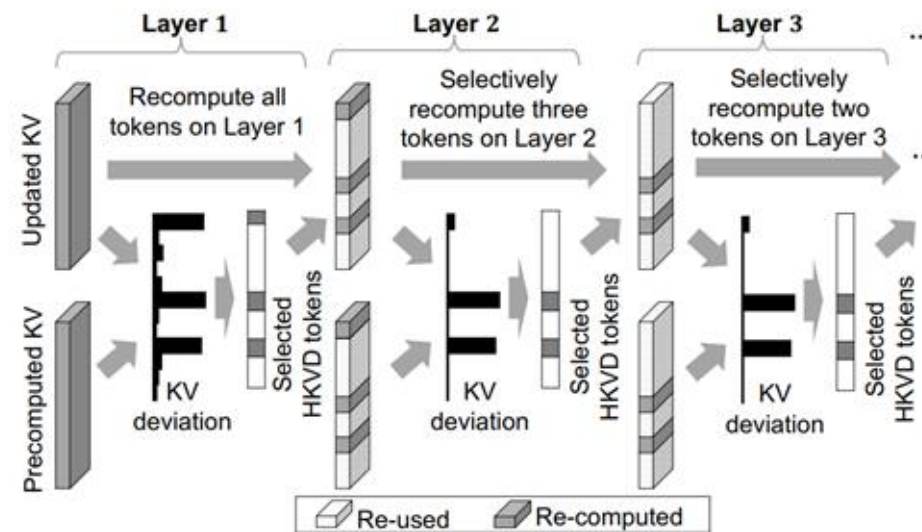
High Spearman rank correlation across consecutive layers on all 3 models

Gradual Filtering

L1: recompute all tokens, rank by Δ_{kv} , pick top $r1\%$ as HKVD

L2+: recompute only HKVD tokens, re-rank, pick $r2\%$ ($r2 < r1$)

Converges to tokens with consistently high deviation across layers.



The Core Insight: Hide Recompute Delay Behind KV Loading

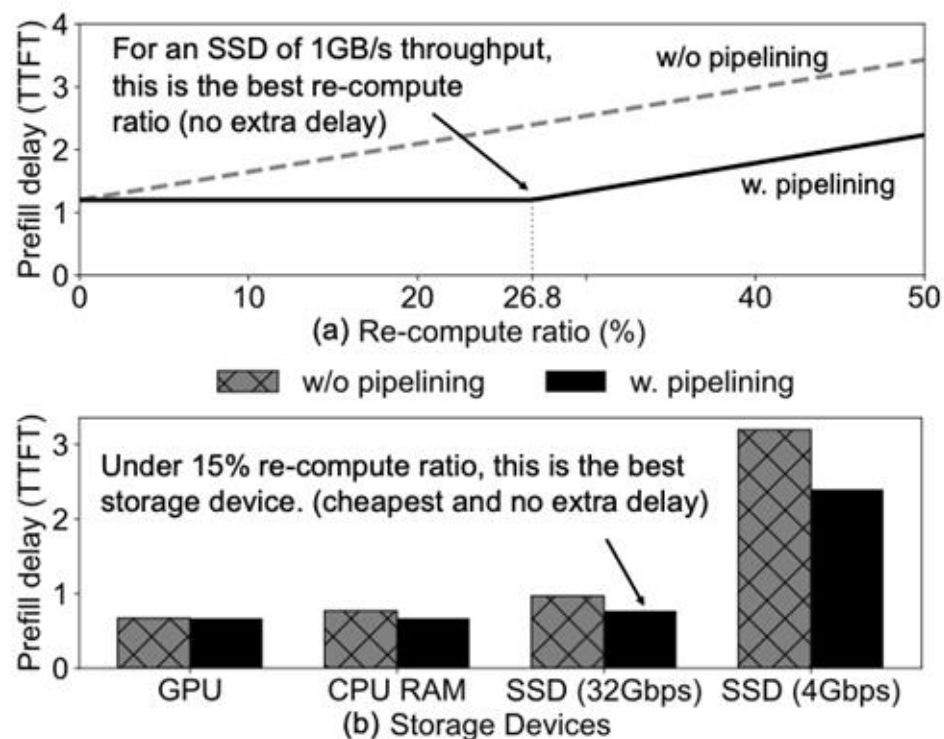


Figure 10. (a) *Smartly picking the recompute ratio will not incur an extra delay.* (b) *Smartly picking storage device(s) to store KV's saves cost while not increasing delay.*

Key Invariant

$$T_{\text{recompute}}(r\%, \text{LLM}, L) \leq T_{\text{load}}(\text{LLM}, L, \text{device})$$

If recompute finishes before KV loading completes, the extra compute is fully hidden.

Concrete Example: Llama-7B, 4K Context

Recompute 15% tokens: **3 ms/layer**

SSD load (NVME): **16 ms/layer**

Component 1: The Loading Controller

Two Optimization Problems in Practice

Problem A: Optimal Recompute Ratio

Given a fixed storage device, find $r\%$ such that:

$$T_{\text{recompute}}(r\%, \text{LLM}, L) = T_{\text{load}}(\text{LLM}, L, \text{dev})$$

$$\text{Then: } r = \max(r\%, r^*\%)$$

where $r^* = 15\%$ is the empirical quality floor, ensuring generation quality remains within tolerance of full prefill.

Problem B: Storage Device Selection

Given $r^* = 15\%$, pick cheapest device where:

$$T_{\text{recompute}} \geq T_{\text{load}}$$

Cost estimator evaluates:

$$C_{\text{store}}(\text{LLM}, L, T, \text{device})$$

Hierarchy: GPU > CPU RAM > SSD (32Gbps) > SSD (4Gbps). Choose cheapest that hides recompute.

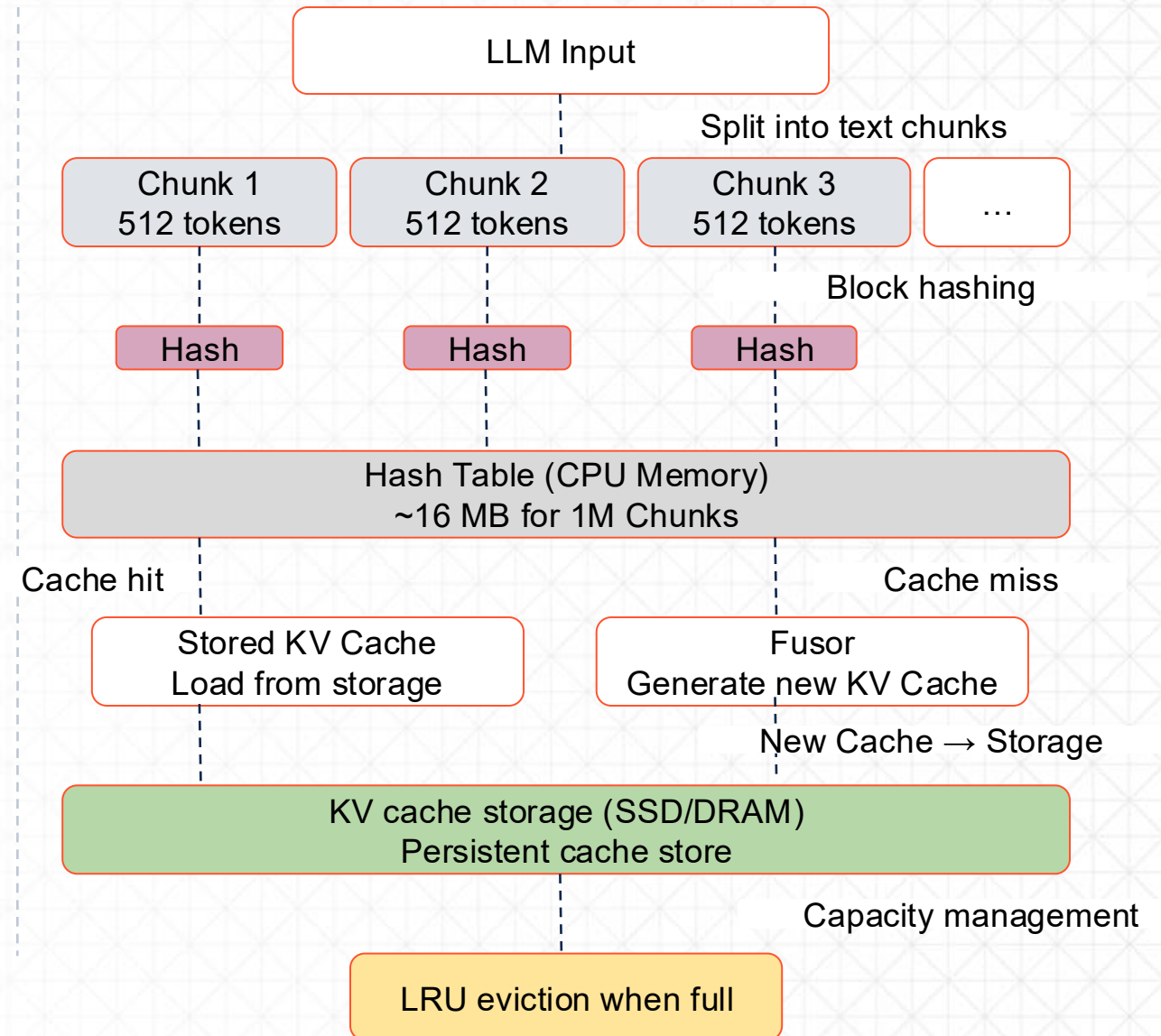
Key Insight: Even with the fastest storage (CPU RAM), the delay is lower-bounded by the 15% minimal recompute to guarantee quality.



Component 2: KV Cache Store

KV Cache Store

1. Split LLM input into text chunks (application-specific, e.g., 512-token RAG chunks).
2. Hash each chunk via block hashing (same mechanism as vLLM) to locate stored KV caches.
3. New chunks' KV caches (generated by Fusor) are added to storage devices.
4. LRU eviction when storage is full. Hash tables kept in CPU (~16 MB for 1M chunks).



Component 3: KV Cache Fusor

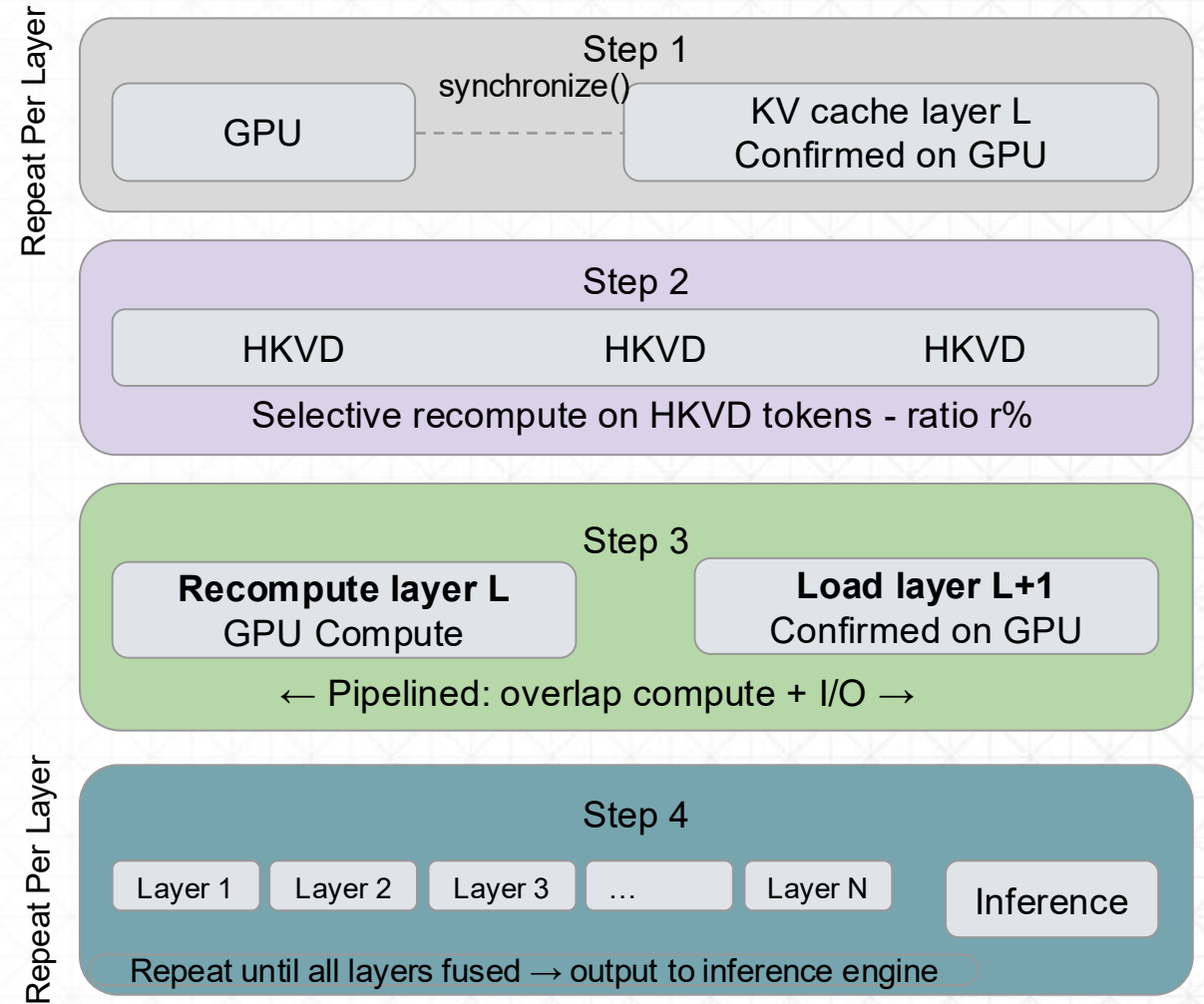
KV Cache Fusor

Step 1: Wait for `synchronize()` to confirm KV cache for layer L is on GPU.

Step 2: Perform selective recompute on HKVD tokens for layer L using ratio $r\%$.

Step 3: Pipeline: while recomputing layer L, begin loading layer L+1 KV cache.

Step 4: Repeat until all layers are fused. Output fused KV cache to inference engine.



Full System Architecture

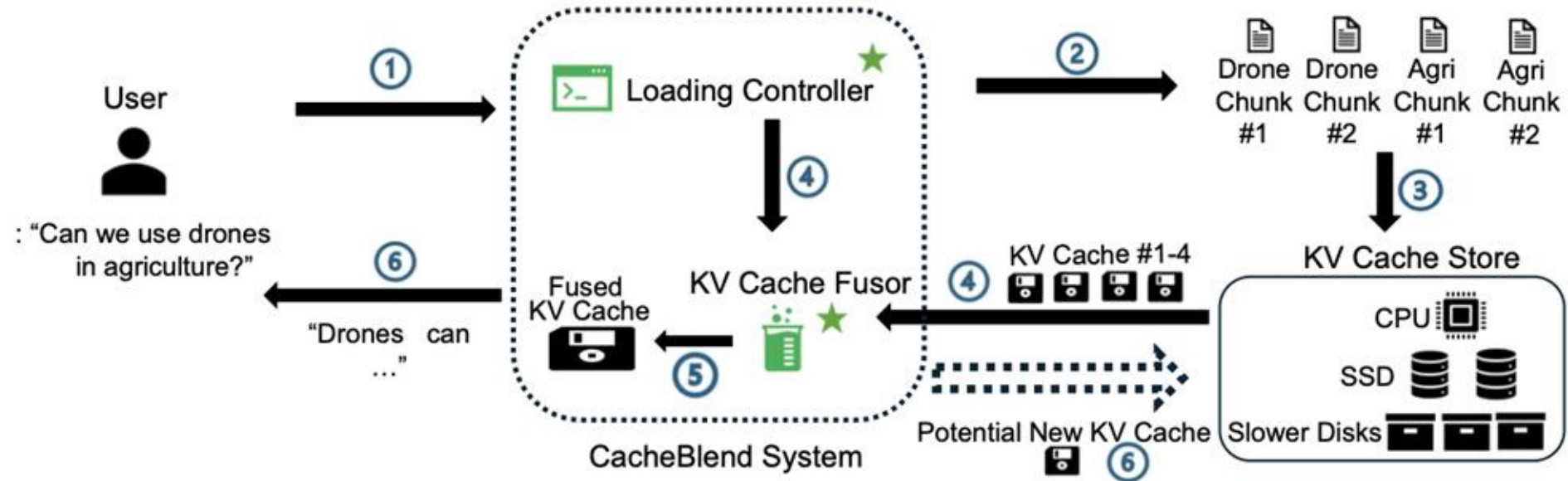


Figure 11. *CACHEBLEND* system (green starred) in light of LLM context augmented generation for a single request. *CACHEBLEND* uses text provided by the retriever, interacts with the storage device(s), and provides KV cache on top of LLM inference engines.

1

User query → retriever fetches relevant text chunks

2

Loading Controller checks KV store (CPU/SSD/Disk)

4

KV Cache Fusor receives caches + computes selective recompute

5

Fused KV cache → LLM inference engine → answer to user

6

Implementation: Built on vLLM

fetch_kv(text, layer_id)

Fetches precomputed KV cache from KV store into GPU. Uses hash lookup, then torch.load() (disk) or torch.cuda() (CPU RAM).

prefill_layer(input_dict, KVCache)

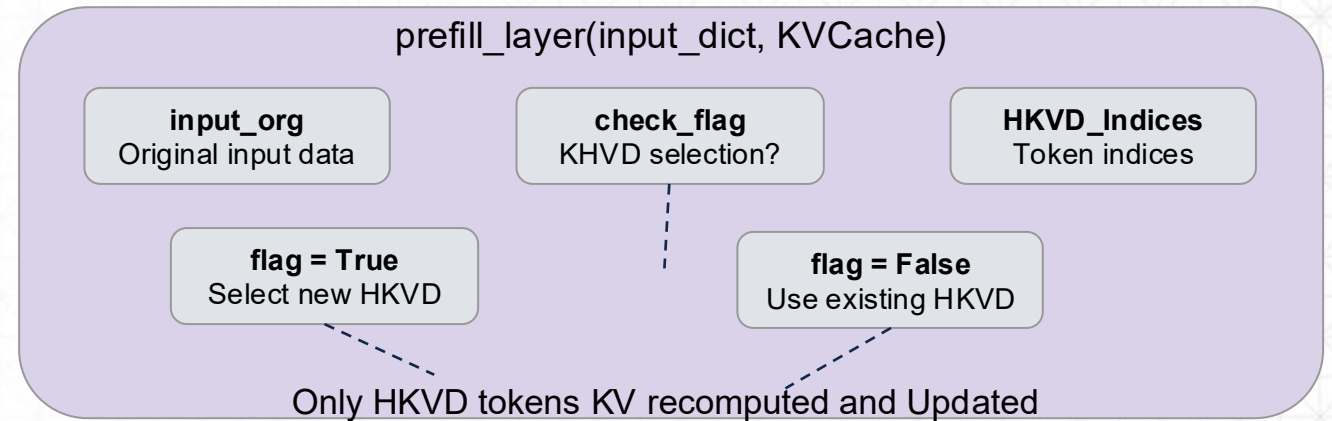
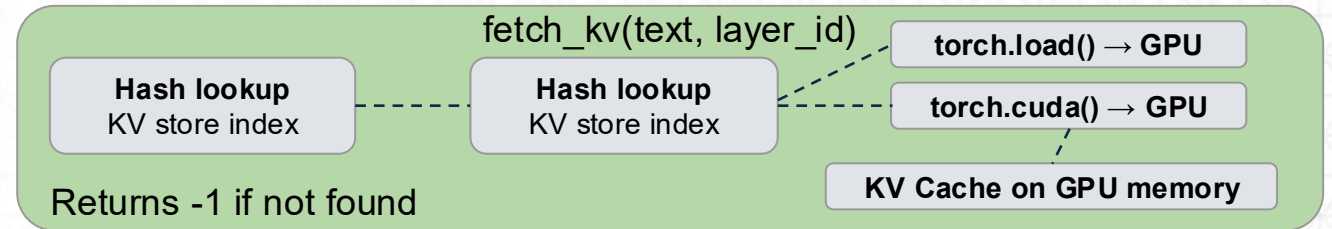
Partial prefill for one layer. Tracks input_org, check_flag (HKVD selection), and HKVD_indices. Only HKVD tokens are recomputed.

synchronize()

Barrier before each prefill_layer ensuring KV cache is loaded. Two threads pipeline layer i compute with layer i+1 loading.

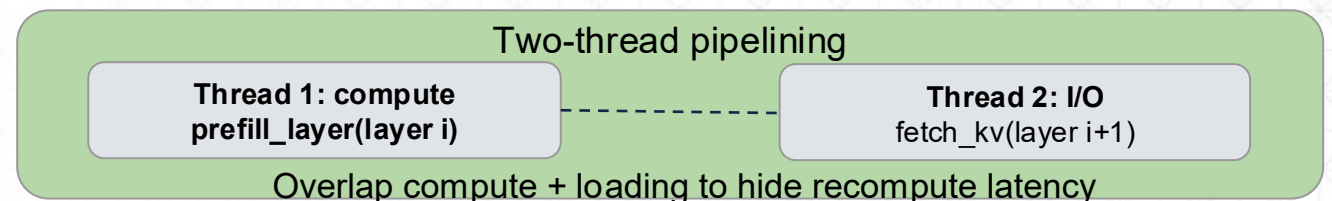
KV Management: New KV caches moved to CPU (torch.cpu) and written to disk in background (torch.save).

vLLM inference engine



synchronize()

Barrier before each prefill_layer – ensures KV cache for this layer is on GPU



Experimental Setup: Models, Datasets & Baselines

Models & Hardware

Mistral-7B (1x A40 GPU, fp16)
Yi-34B (1x A40, 8-bit quantized)
Llama-70B (2x A40, 8-bit quantized)
128 GB RAM - 1TB NVMe SSD (4.8 GB/s)

Dataset & Metrics

2WikiMQA - multi-hop QA (F1-score)
Musique - multi-hop QA (F1-score)
SAMSum - dialogue summary (Rouge-L)
MultiNews - news summary (Rouge-L)

Baselines

Full KV Recompute

Full prefill on every request - correct but slowest

Prefix Caching (SGLang)

Only 1st chunk, idealized with no load delay

Full KV Reuse (PromptCache)

All caches reused, ignores cross-chunk attention

MapReduce / MapRerank

LangChain RAG methods, process chunks independently



Main Result: TTFT Reduction with Negligible Quality Drop

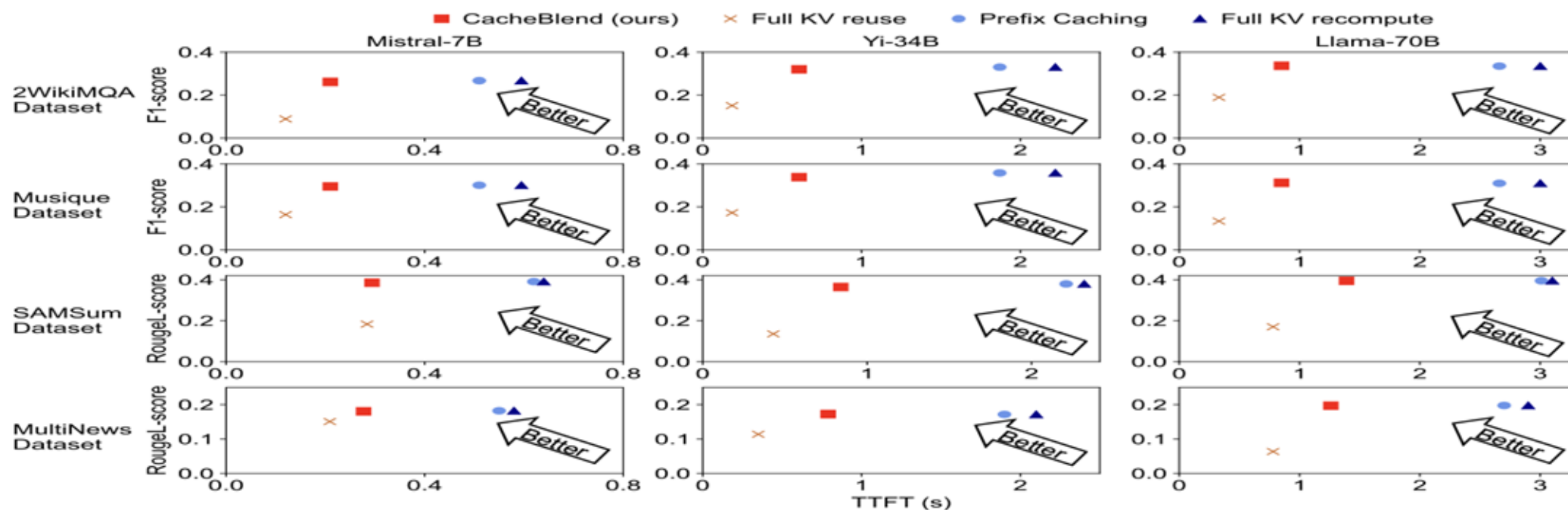


Figure 12. *CACHEBLEND* reduces TTFT by 2.2–3.3× compared to full KV recompute with negligible quality drop across four datasets and three models.

2.2–3.3×

TTFT reduction
vs. full recompute

≤ 0.02

F1 / Rouge-L quality
drop vs. recompute

+0.1–0.2

F1 gain over
full KV reuse

Current Limitations

Transformer-Only

Insights (e.g., HKVD correlation across layers) apply only to transformer architectures. Non-transformer models like Mamba (SSM) and Griffin (gated linear recurrence) are not yet supported.

Limited Quantization Testing

Evaluation uses 8-bit quantization for Yi-34B and Llama-70B but has not explored other quantization settings or their interaction with selective recompute.

Single Serving Engine

Implemented on vLLM only. Not yet tested on newer engines like DistServe or StableGen that disaggregate prefill and decode.

Future Directions

Cross-Node KV Sharing

Extend CacheBlend to distributed settings where KV caches are shared across compute nodes for multi-GPU serving.

Novel Serving Engines

Integrate with DistServe's prefill-decode disaggregation and StableGen's chunked-prefill piggybacking for compounding savings.

Non-Transformer Architectures

Investigate selective recompute equivalents for state-space models (Mamba) and hybrid architectures (Griffin).

AVA: Towards Agentic Video Analytics with Vision Language Models

Presented By: Smit Vasani, Ram, Sreeja



Why Video Analytics Matters



Surveillance

24/7 security monitoring



Wildlife

Behavior over weeks



Traffic

Intersection analytics



Industrial

Factory automation

Current Systems (L1–L3)

“Count pedestrians” “Detect cars”
Predefined tasks only

THE GAP

100s of hours of video
Context window ≤ 1 hr
No open-ended queries

What We Need (L4)

“What caused the accident?”
Open-ended reasoning at scale

Video Analytics Intelligence: L1 – L5

Closed-End

Open-End, Agentic →

L1

**Basic
Perception**

Classification,
detection, segmentation

L2

**Event
Detection**

Causal events,
action recognition

L3

**Language
Query**

NLP queries,
domain-specific

L4

**Open-ended
Reasoning**

Complex reasoning,
long-term causal

L5

**Knowledge-
Augmented**

External knowledge,
autonomous insights

↑ **AVA's Target**

Future Work



The Problem with VLMs

✓ What VLMs Can Do

- ✓ **Open-ended queries**
Natural language interaction with video
- ✓ **Causal reasoning**
“Why did X happen?” “What caused Y?”
- ✓ **Zero-shot understanding**
No task-specific training needed

✗ The Context Window Problem

768

frames max
≈ 1 hour

✗ **Real-world: 100s of hours**
Wildlife, traffic, surveillance

✗ **Cost grows linearly**
Longer video =
proportionally more
expensive

Fundamental gap: VLMs handle minutes → Video analytics needs hundreds of hours

0.1–0.5%

of video frames are needed to answer any given question

VideoMME Benchmark Experiment with Qwen2-VL

Short (1.4 min)

12 needed (0.5%)

Medium (9.7 min)

68 needed (0.4%)

Long (39.7 min)

82 needed (0.1%)

Why Existing RAG Approaches Fall Short



Vectorized Retrieval

✓ How it works

Embed each frame with CLIP, match query embedding to retrieve similar frames

✗ Fails on

Summaries (“What happened?”) and multi-hop queries (“What did he do after X?”)

e.g. CLIP-based frame retrieval



Video Structuring

✓ How it works

Extract ASR, OCR, objects with specialized tools, then apply text-based RAG

✗ Fails on

Can’t predict needed tools in advance; limited to pre-extracted info types

e.g. Video-RAG (ASR + OCR + detection)



Iterative Retrieval

✓ How it works

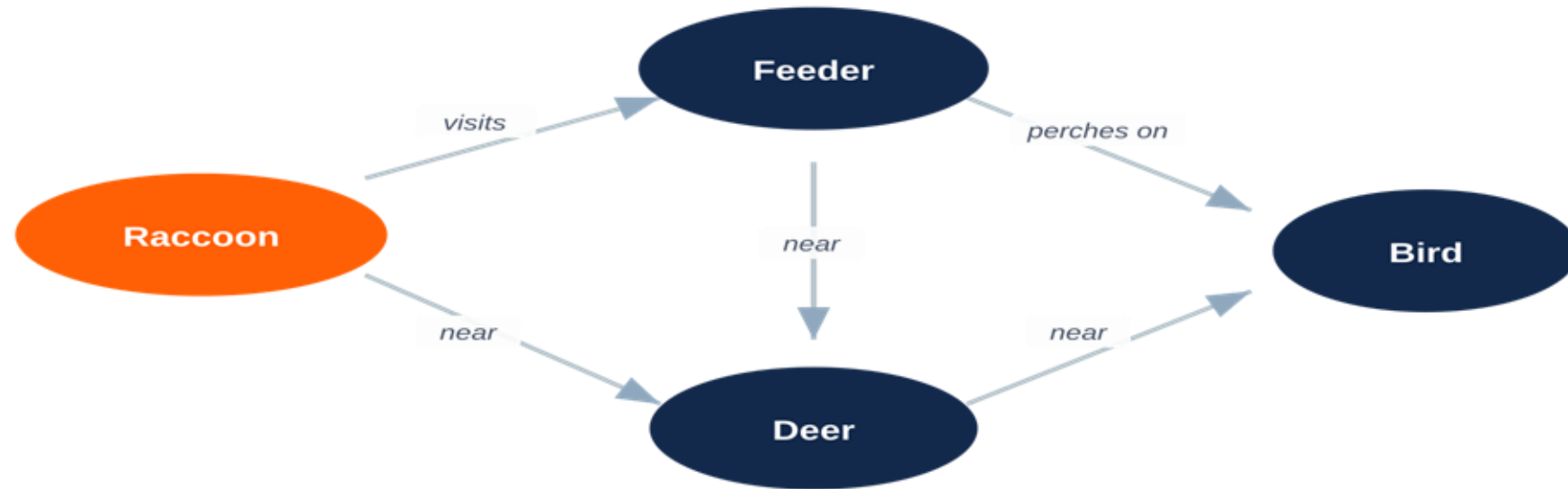
Coarse sample → VLM picks segments to retrieve next → iterate

✗ Fails on

Initial sampling misses info in long videos; cost grows with duration

e.g. VideoAgent, VideoTree, VCA

Background: What is a Knowledge Graph?



Wildlife monitoring scenario — static entity relationships only

Nodes = Entities

People, animals, objects, locations

Edges = Relationships

Directed links like “visits”, “near”

Static Snapshot

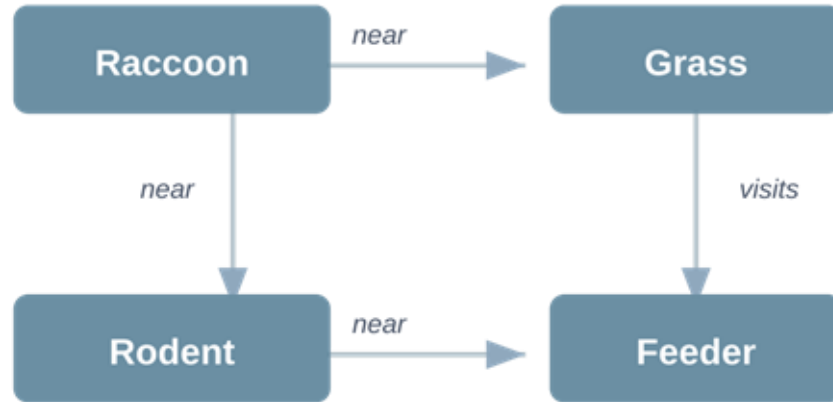
No temporal order, no event flow

Used in Text RAG

GraphRAG, LightRAG, MiniRAG

Event Knowledge Graph vs. Knowledge Graph

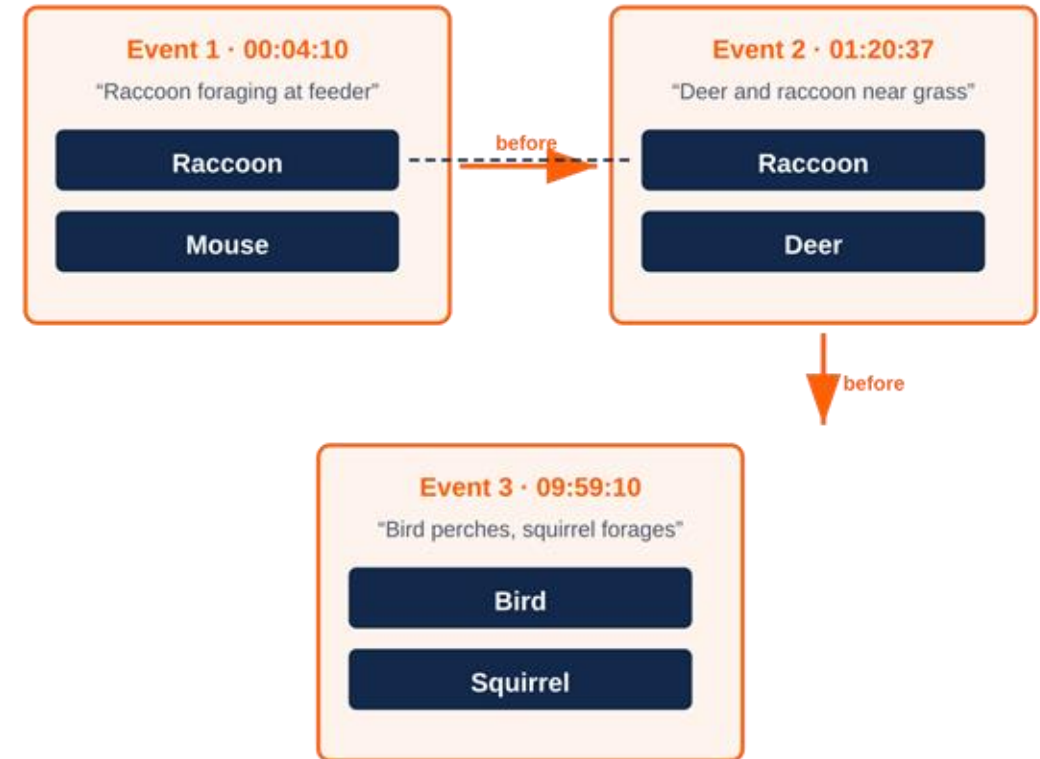
Knowledge Graph (KG)



X No temporal order, no event context

Flat entity-relationship structure

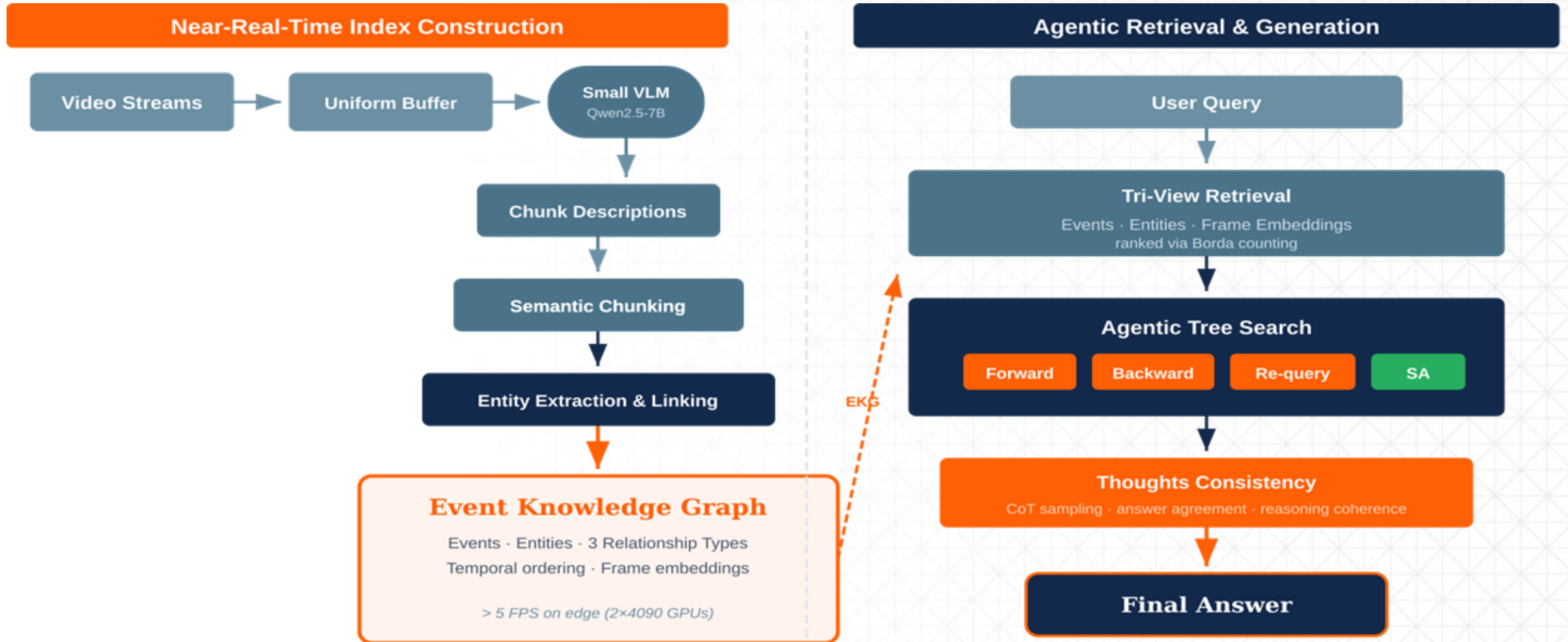
Event Knowledge Graph (EKG)



Temporal ordering · Entity linking across events · Event descriptions

AVA System Overview

Build an EKG index in real-time with a small VLM → Retrieve & generate answers via agentic tree search

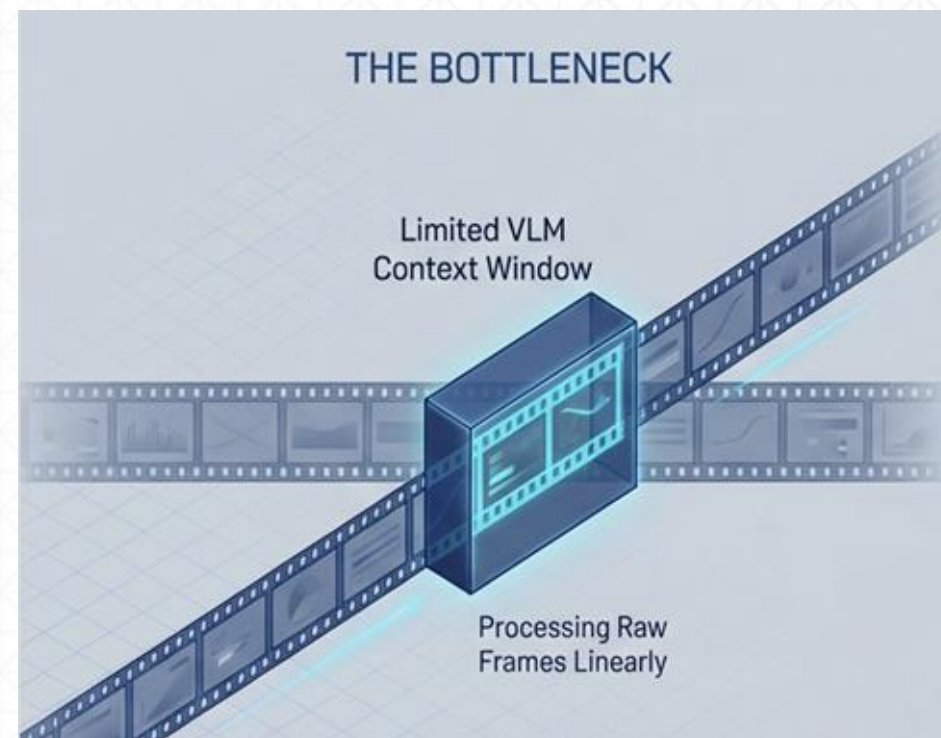


Near-Real-Time Index Construction



Escaping the content window limitation

- Instead of processing raw frames linearly, AVA restructures frames into a navigable, spatial memory index called Event Knowledge Graph in near real time.
- An EKG is a structured representation of events and their interconnections, linking entities, timestamps, locations, and other contextual information



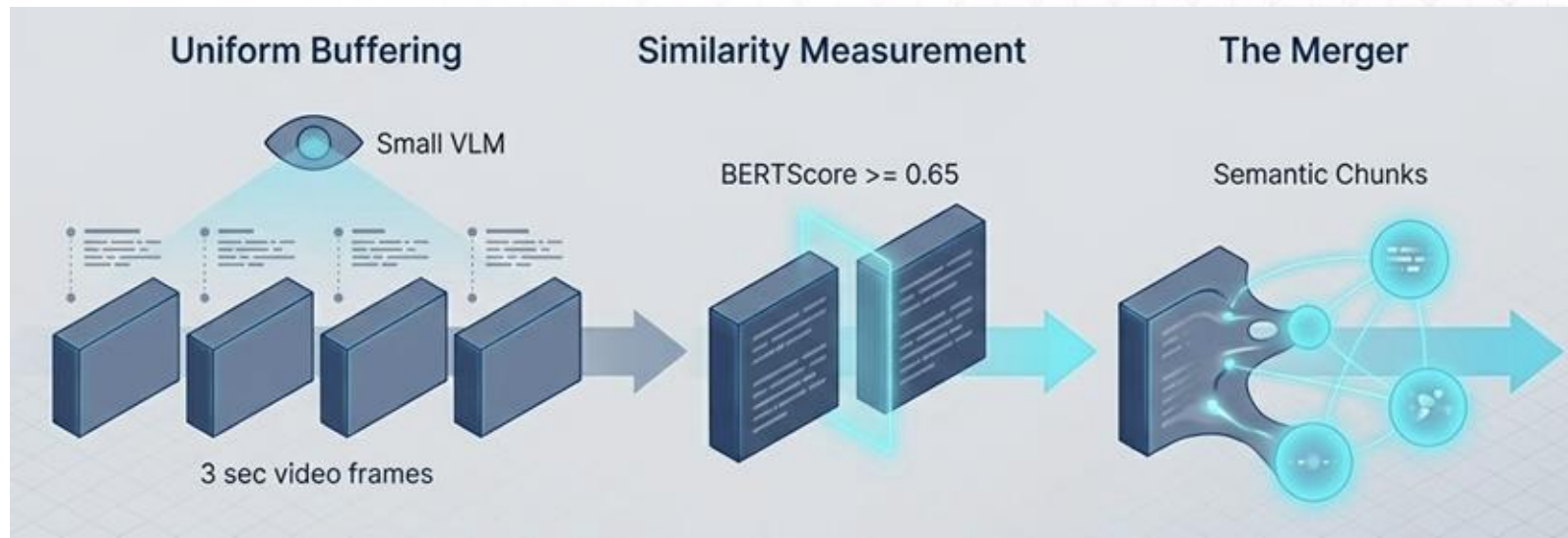
Why is chunking needed?

- Large VLMs can achieve high accuracy in event detection and transcription, but their computational overhead makes it difficult to process video streams in near-real-time.
- Small VLMs offer reduced latency but suffer from performance degradation as the length of the video increases.
- Both large and small VLMs are limited by their constrained context windows.



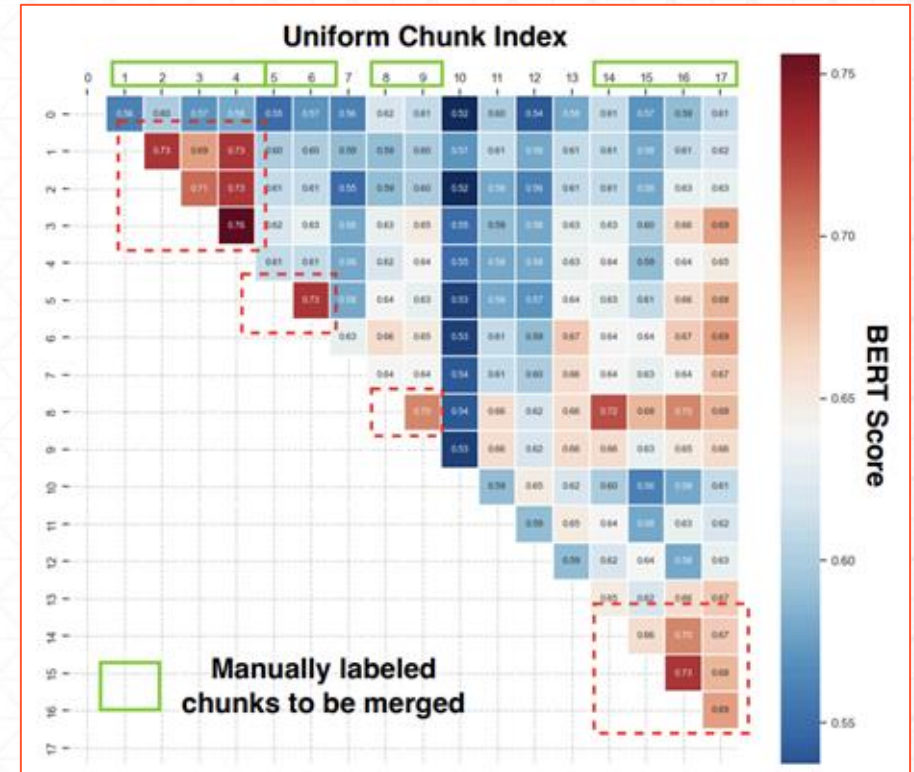
Semantic Chunking

- Split the video into fixed 3-second segments and use a vision-language model like Qwen2.5-VL-7B to generate concise event descriptions for each segment.
- Convert these descriptions into embeddings using a model such as BERT and compute similarity between neighboring segments.



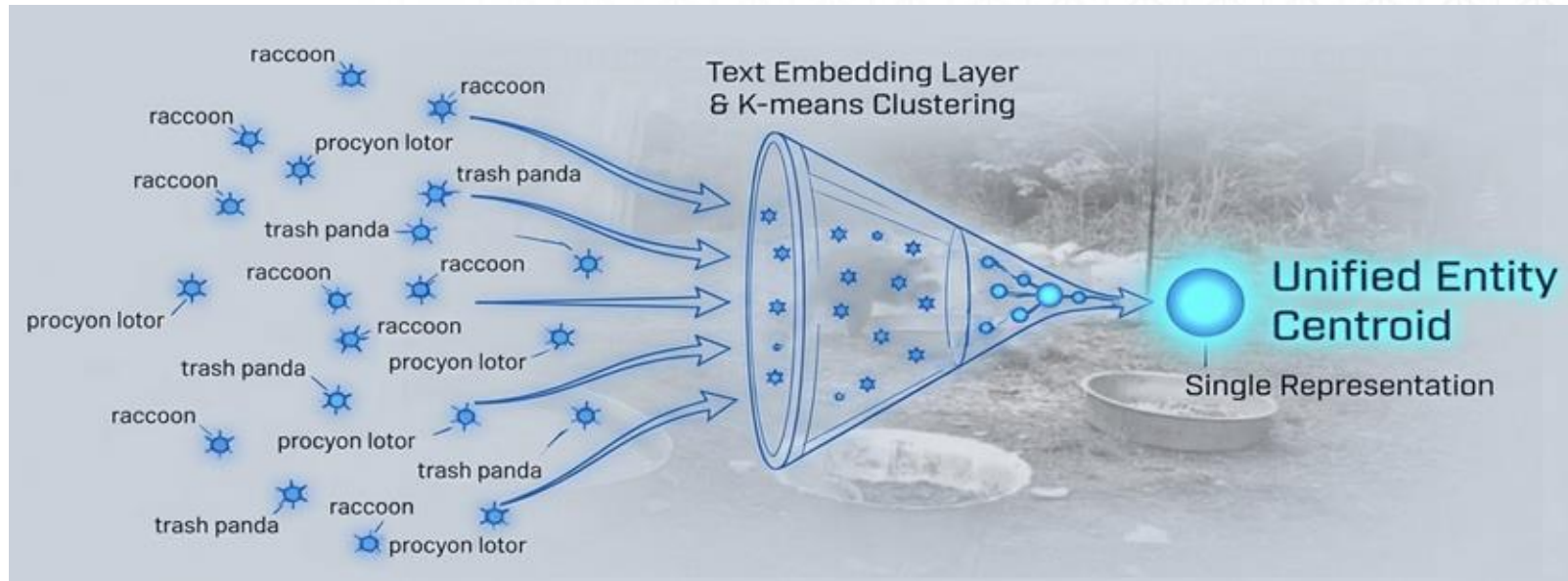
Merging Related Frames

- Merge chunks into semantic segments based on thresholds: high similarity within a segment and low similarity at segment boundaries.
- Summarize each merged segment using the VLM, with parallelized similarity computations ensuring efficient, near real-time processing.



Entity Extraction & Linking

- Encode extracted entities using an embedding model like JinaCLIP and cluster them via K-means to de-duplicate and link semantically similar entities (e.g., “raccoon” and “procyon lotor”).
- Store the resulting structured knowledge graph (events, entities, and relationships) along with vectorized video frames to enable efficient and comprehensive retrieval.



Agentic Retrieval & Generation



Tri-View Retrieval

- AVA retrieves information from three views, events, entities, and raw video frames, by encoding the query and matching it using embeddings from JinaCLIP.
- The results are linked through the knowledge graph and aggregated using a weighted Borda count to rank the most relevant events.

$$s_m(e_j) = \frac{\text{sim}_m(e_j)}{\sum_{e_k \in \mathcal{E}_m} \text{sim}_m(e_k)},$$

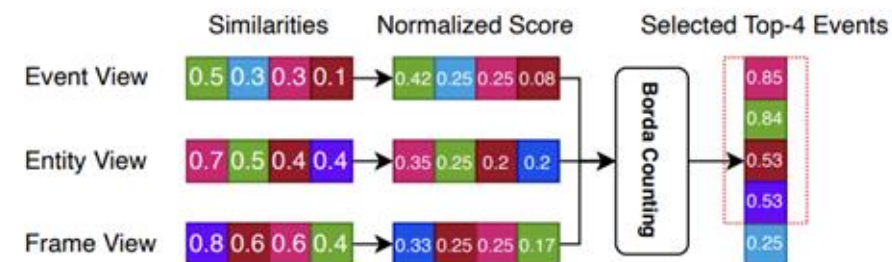
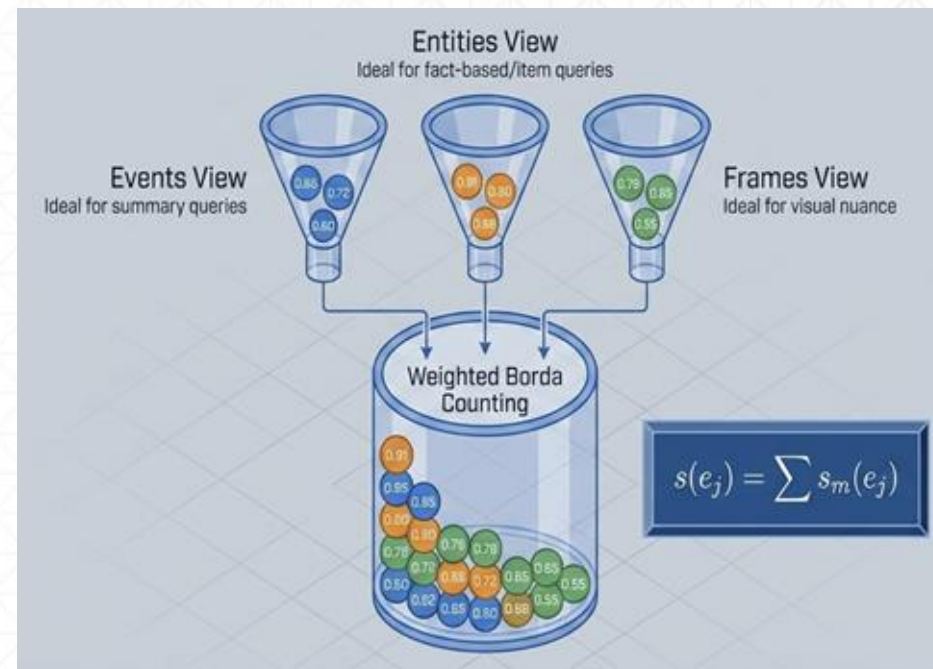


Figure 5: An illustration of tri-view retrieval and borda counting on the retrieved events.

Agentic Searching on a Graph

Forward

Extends retrieval by adding future events linked in the EKG. Helps understand how the situation evolves over time.



Like humans anticipating what happens next to understand the progression of a story.

Backward

Retrieves earlier events to provide historical context and causal background. Enables tracing the narrative back in time.



Similar to humans looking back at past events to understand context and causes.

Re-Query

Generates new keyword-based queries using an LLM to fetch additional relevant events. Expands retrieval from different perspectives.



Mirrors how humans reframe questions or seek new angles when initial information is insufficient.

S&A

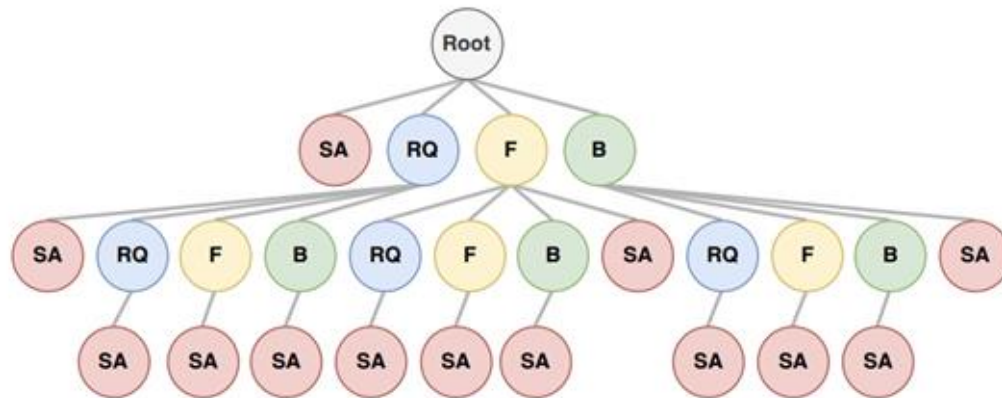
Uses retrieved event descriptions to generate a final response with an LLM. Synthesizes information into a coherent answer.



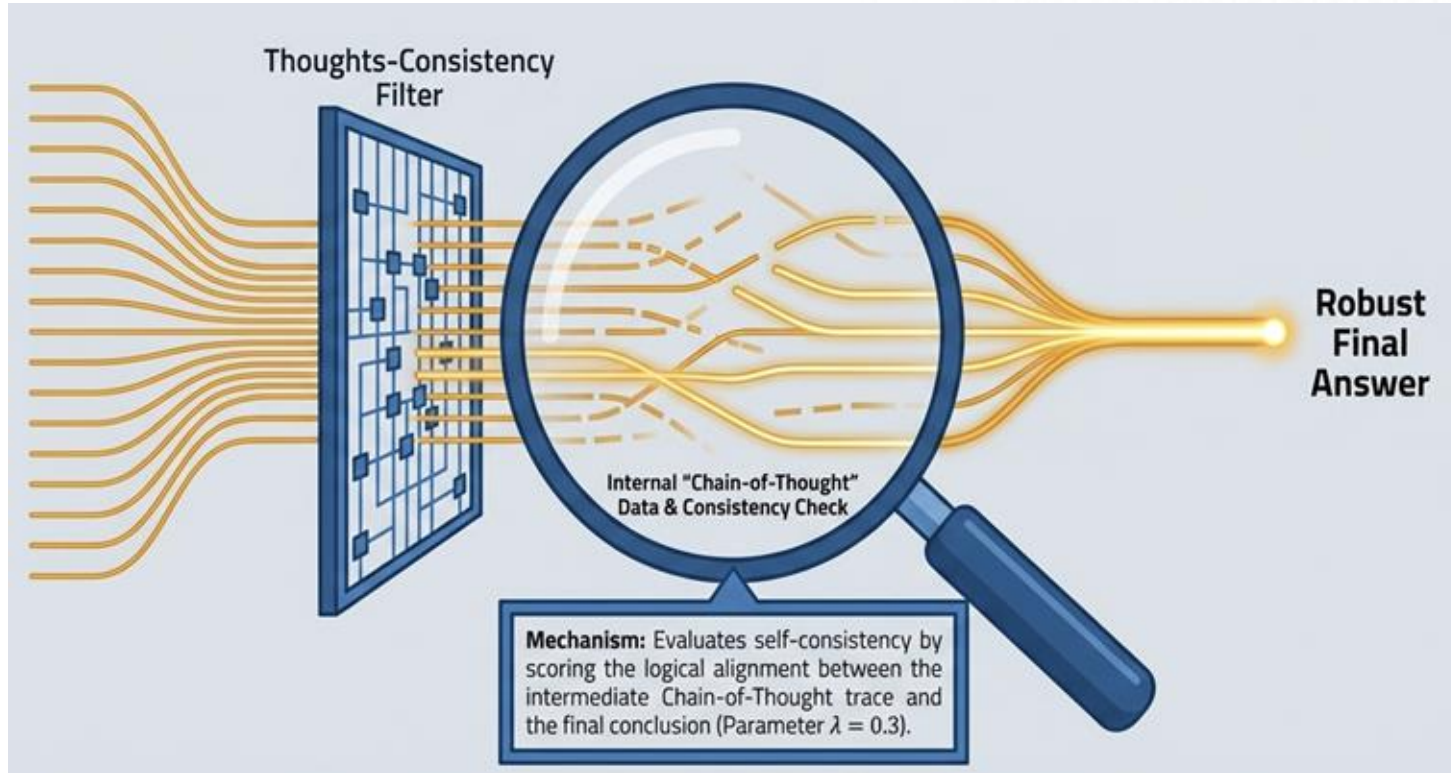
Reflects how humans synthesize gathered information into a clear, concise conclusion.

Agentic Searching on a Graph

- For each node, a single-step rollout is conducted in which four predefined actions, namely, forward, backward, re-query, and summary and answer, are executed on every events on the node
- This tree search mechanism systematically explores multiple pathways to extract information from the EKG



Consistency Enhanced Generation



$$S_a^{(t)} = \frac{|\{i \mid a_i = a^{(t)}\}|}{n}$$

$$S_r^{(t)} = \frac{2}{k(k-1)} \sum_{1 \leq i < j \leq k} \text{BERTScore}(r_i, r_j),$$

k is the number of times $a^{(t)}$ appears in the n samples.

$$S_{\text{final}}^{(t)} = \lambda S_a^{(t)} + (1 - \lambda) S_r^{(t)},$$

Check and Answer (CA)

01

Select Top-2 Conflicting Nodes

After ranking all SA candidates, identify the top-2 nodes with differing answers. These are the candidates to verify.

02

Retrieve Raw Video Frames

Extract the actual video frames associated with EKG events linked to these conflicting nodes.

03

VLM Re-evaluation

Prompt the VLM to regenerate an answer by directly attending to the visual frames, catching anything missed during graph construction.

Implementation

EKG Construction

Qwen2.5-VL-7B

Small VLM · runs on edge server

- Describes 3-second video chunks in real-time
- Extracts entities and relationships per event
- Accelerated with AWQ quantization + LMDeploy
- Batch inference maximizes GPU utilization
- Achieves >5 FPS on 2×4090 (input is 2 FPS)

Agentic Search(SA)

Qwen2.5-32B

Large LLM · reasoning & answer drafts

- Drives tree-search: Forward / Backward / Re-query / SA
- Generates Chain-of-Thought answers at SA nodes
- Qwen2.5-14B tested as lighter alternative
- LMDeploy + AWQ deployed on A100 GPU
- Main latency bottleneck: 101–174 seconds/query

Frame Verification (CA)

Gemini-1.5-Pro

Cloud VLM · visual double-check

- Looks at actual raw video frames (not just EKG text)
- Confirms or corrects the top SA candidate answer
- API-based: only 14.2s latency
- Qwen2.5-VL-7B used as on-device fallback
- Catches errors made during EKG description phase

Evaluation: Benchmarks & Baselines

LVBench

- 103 videos · 1,549 questions
- ~4,100 sec (~68 min) avg duration
- 6 domains · 6 task types

Temporal grounding, summarization, reasoning, entity recognition, event understanding, KIR

VideoMME-Long

- 300 videos · 900 questions
- ~2,400 sec (~40 min) avg duration
- 6 domains · 12 task types

Broad scenario coverage with 30 subfields. Videos > 20 min only.

AVA-100 (Proposed by Authors)

- 8 videos · 120 QA pairs
- >10 hrs each (~99 hrs total)
- 4 real-world scenarios

Daily life · City walking · Wildlife monitoring · Traffic monitoring - human-annotated Q&A

Baselines Compared Against

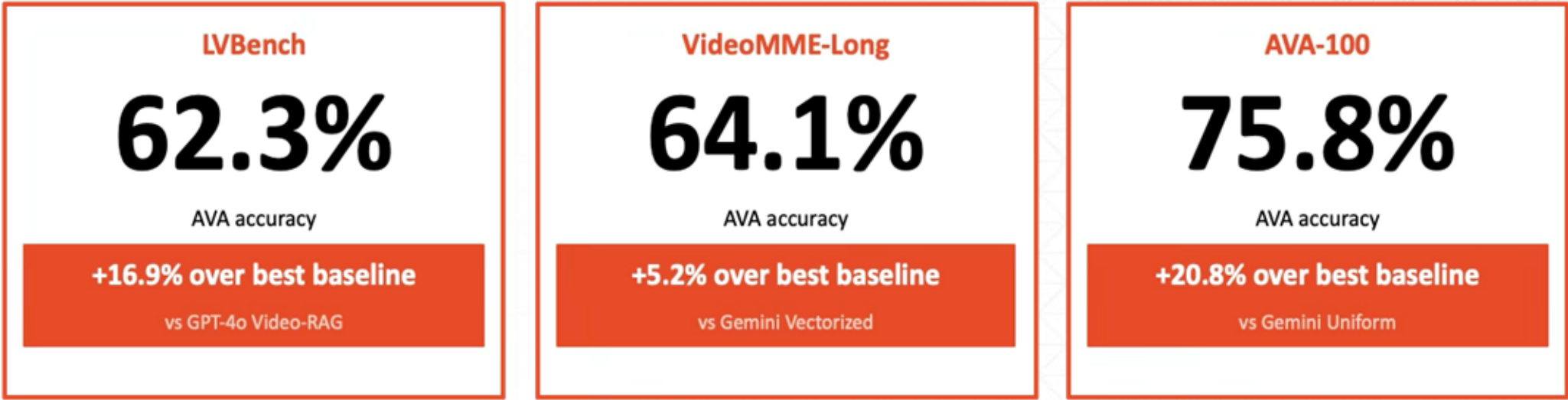
VLM Baselines (each tested with uniform sampling AND vectorized retrieval)

GPT-4o · Gemini-1.5-Pro · Phi-4-Multimodal · Qwen2.5-VL-7B · InternVL2.5-8B · LLaVA-Video-7B

Video-RAG Systems (all built on GPT-4o / GPT-4)

VideoTree (GPT-4o) · VideoAgent (GPT-4o) · VCA (GPT-4o) · DrVideo (GPT-4)

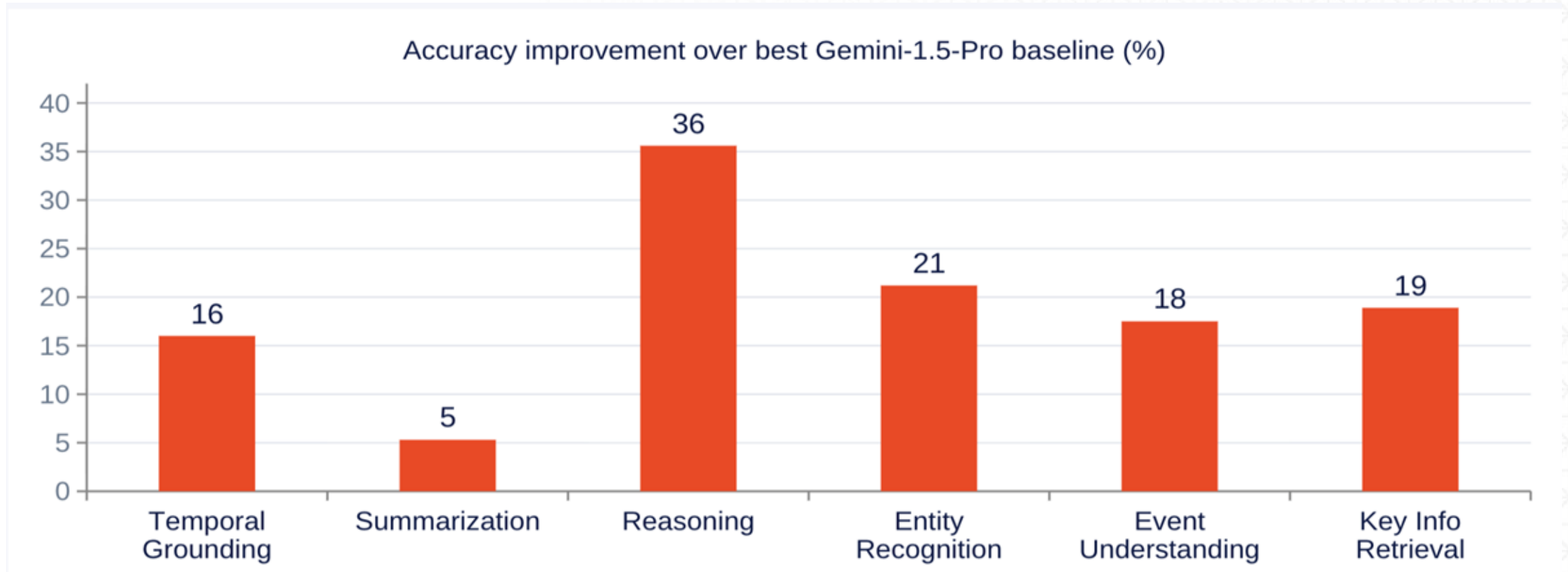
Overall Performance Results



Detailed Improvements by Comparison Type

Comparison	LVBench	VideoMME-Long	AVA-100
vs Video-RAG systems (GPT-4o)	+21.0%	+7.8%	-
vs Vectorized retrieval (best VLM)	+16.9%	-	+20.8%
vs Uniform sampling (best VLM)	+19.6%	-	+26.9%

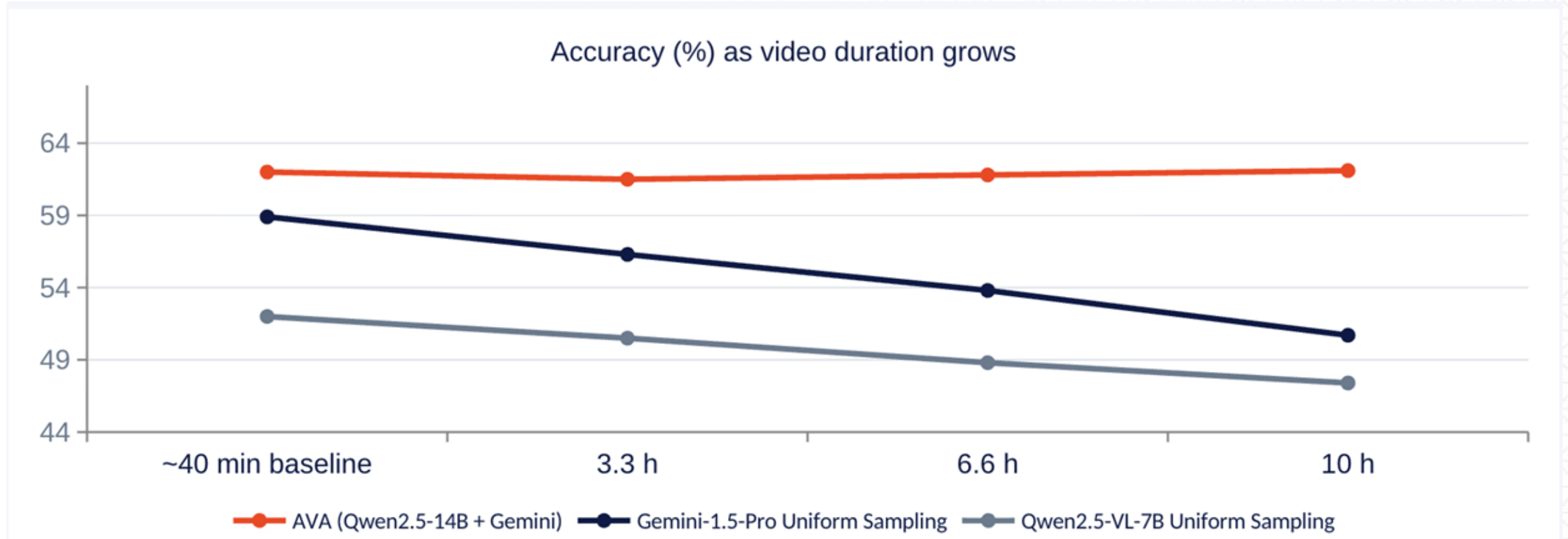
Per-Task Accuracy on LVBench



Reasoning sees the largest gain (+35.6%) because AVA's Forward/Backward agentic actions can traverse EKG temporal links to connect cause-and-effect events hours apart - something uniform sampling and CLIP retrieval fundamentally cannot do.

Scalability: Performance vs. Video Length

Experiment: videos concatenated from VideoMME-Long to create 3.3h, 6.6h, and 10h sequences. Same questions used at each length.



- Baselines degrade significantly: Gemini drops – 8.2% at 10h (uniform), Qwen drops – 4.6%
- AVA stays flat (< 0.5% variation) – EKG scales independently of video length
- Computational overhead of AVA does NOT grow with video duration

System Overhead

Index Construction Speed (FPS)

Input stream = 2 FPS · Must process faster than input to stay real-time



Query Answering Latency (per query)

Stage	Model	Time
Tri-View Retrieval	JinaCLIP	0.44 s
Agentic Search	Qwen2.5-14B	101.5 s
Agentic Search	Qwen2.5-32B	174.2 s
Consistency (CA)	Qwen2.5-VL-7B	45.8 s
Consistency (CA)	Gemini-1.5-Pro	14.2 s

⚠ **Bottleneck:** Agentic search (LLM inference) is the dominant latency at 101–174 seconds. This is the primary target for future optimization - replacing brute-force Monte Carlo search with a learned policy model.

Limitations and Future Work

- **Fixed Monte Carlo Tree-Search Strategy**

Limitation: The existing agentic retrieval and generation mechanism relies on a fixed tree-search strategy based on the Monte Carlo approach. While effective, this method is computationally expensive, with overhead growing significantly with search depth.

Future Work: Use the collected search trajectories as training data to develop a model capable of dynamically selecting optimal search actions and depths based on the query and EKG context.

- **Specialized Visual Tasks (e.g. Exact Counting)**

Limitation: The integrated VLM may encounter challenges in certain specialized visual tasks, such as precise object counting.

Future: Integrate lightweight, task-specific vision models as callable tools within the system, enabling the VLM to function as an autonomous agent that intelligently invokes these specialized tools to handle tasks like precise object counting.